this Keyword

"This" keyword refers to an object that is executing the current piece of code.

```
const student = {
  name: "nithin",
  age: 23,
  eng: 95,
  math: 93,
  phy: 97,
  getAvg() {
    let avg = (this.eng + this.math + this.phy) / 3;
    console.log(avg);
  }
};
```

try & catch

The try statement allows you to define a block of code to be tested for errors while it is being executed.

The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

```
console.log("hello");
console.log("hello");
console.log(a);
console.log("hello2");
console.log("hello2");
console.log("hello2");
```

```
try {
    console.log(a);
} catch {
    console.log("variable a doesn't
}
```

Arrow Functions

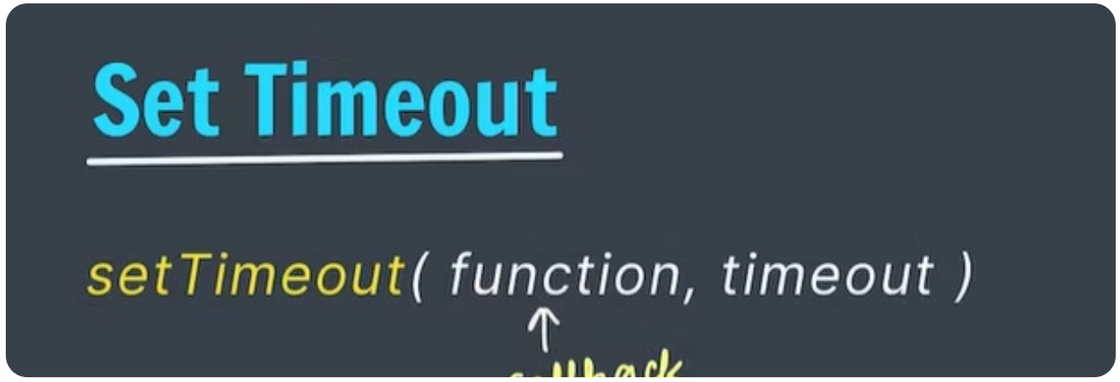
const func = (arg1, arg2 ..) => { function definition }

```
const sum = (a, b) => {
    console.log(a+b);
}
```

```
const sum = (a, b) => {
  console.log(a + b);
};

const cube = (n) => {
  return n * n * n;
};

const pow = (a, b) => {
  return a ** b;
};
```



It only schedules the function to run later, then JavaScript continues to the next line immediately.

```
console.log("hi there!");

setTimeout(() => {
  console.log("opqtech");
}, 4000);

console.log("welcome to");
```

After 4000ms, run it when the main thread is free.

Set Interval

setInterval(function, timeout)

It runs `func` again and again, with at least `time` milliseconds between scheduled runs.

```
const id = setInterval(() => {
  console.log("tick");
}, 1000); // every ~1 second
```

`setInterval` returns an id. Use `clearInterval(id)` to stop.

```
const student = {
  name: "aman",
  marks: 95,
  prop: this, //global scope
  getName: function () {
    console.log(this);
    return this.name;
  },
  getMarks: () => {
    console.log(this); //parent's scope -> window
    return this.marks;
  },
  getInfo1: function () {
    setTimeout(() => {
      console.log(this); //student
    }, 2000);
  },
  getInfo2: function () {
    setTimeout(function () {
      console.log(this); //window
    }, 2000);
```

061nithin@gmail.co

this in Functions vs Arrow Functions

Regular Function (function keyword)

- this depends on who calls the function (runtime binding)
- Called as a method, this equals the object
- Called standalone, this equals window (or undefined in strict mode)

Arrow Function (=>)

- this is lexically inherited from the surrounding scope at the moment the arrow is created
- It does NOT get its own this
- No matter how it's called, this stays the same

From the example:

getName is a regular function so this equals student (the caller).

getMarks is an arrow function so this equals window (lexically inherited from global scope where student was defined).

getInfo1 uses an arrow function inside setTimeout. The arrow inherits this from getInfo1, which is student. Result: student.

getInfo2 uses a regular function inside setTimeout. It is called by the browser, so this equals window. Result: window.

Rule of thumb: Use arrow functions when you want to preserve this from the outer scope. Use regular functions when you need this to be determined by the caller.

Array Methods

- forEach
- map
- filter
- some
- every
- reduce

Array Methods — Examples

```
const nums = [1, 2, 3, 4, 5];
```

forEach — Executes a function on each element (no return value)

```
nums.forEach((num) => console.log(num * 2));
// 2, 4, 6, 8, 10
```

map — Returns a new array with transformed elements

```
const doubled = nums.map((num) => num * 2);
// [2, 4, 6, 8, 10]
```

filter — Returns a new array with elements that pass a condition

```
const evens = nums.filter((num) => num % 2 === 0);
// [2, 4]
```

some — Returns true if at least one element passes the condition

```
const hasEven = nums.some((num) => num % 2 === 0);
// true
```

every — Returns true only if ALL elements pass the condition

```
const allPositive = nums.every((num) => num > 0);
// true
```

reduce — Reduces the array to a single value using an accumulator

```
const sum = nums.reduce((acc, num) => acc + num, 0);
// 15
```

Finding Maximum in an array

```
let nums = [2, 3, 4, 5, 3, 4, 7, 8, 1, 2];

let result = nums.reduce( (max, el) => {
    if(el > max) {
        return el;
    } else {
        return max;
    }
});
```

Practice Qs

Check if all numbers in our array are multiples of 10 or not.

Create a function to find the min number in an array.

1) Check if all numbers are multiples of 10

Use `.every()`:

```javascript
const arr = [10, 20, 30, 40];

const allMultiplesOf10 = arr.every(n => n % 10 === 0);

console.log(allMultiplesOf10); // true
```

If any element is not a multiple of 10, it becomes `false`.

---

2) Function to find the minimum number in an array

Option A: Using `.reduce()` (array method)

```javascript
function findMin(arr) {
  return arr.reduce((min, cur) => (cur < min ? cur : min), arr[0]);
}

console.log(findMin([30, 5, 99, -2, 10])); // -2
```

Default Parameters

Giving a default value to the arguments

```
function func (a, b = 2) {
    //do something
}
```

Spread

Expands an iterable into multiple values

```
function func (...arr) {
    //do something
}
```

```
> let arr = [1, 2, 3, 4, 5];
<· undefined
> let newArr = [...arr];
<· undefined
> newArr
<· ▶ (5) [1, 2, 3, 4, 5]
```

```
> let chars = [..."hello"];
<· undefined
> chars
<· ▶ (5) ['h', 'e', 'l', 'l', 'o']
```

```
let odd = [1, 3, 5, 7, 9];
let even = [2, 4, 6, 8, 10];

let nums = [...odd, ...even];
```

```
let data = {
  email: "ironman@gmail.com",
  password: "abcd",
};

let dataCopy = { ...data, id: 123 };
```

Rest

Allows a function to take an indefinite number of arguments and bundle them in an array

```
function sum(...args) {
  return args.reduce((add, el) => add + el);
}
```

Destructuring

Storing values of array into multiple variables

```
let names = ["tony", "bruce", "steve", "peter"];
let [winner, runnerup] = names;
console.log(winner, runnerup); //"tony" "bruce"
```

```
const student = {
  name: "karan",
  age: 14,
  class: 9,
  subjects: ["hindi", "english", "math", "science"],
  username: "karan@123",
  password: "abcd",
};

let { username: user, password: secret } = student;
```